

Tecnologias de Reuso: Engenharia baseada em Serviços

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Dimensões do Reuso
- SOA – Service-Oriented Architecture
- Aplicações e Casos de Uso de SOA
- Arquitetura de Microsserviços
- Os 12 Factors (Princípios de Aplicações Cloud-Native)
- Por que resiliência importa?
- Padrões de Resiliência em Microsserviços
- Microserviço REST (FastAPI) → Conversor de moeda

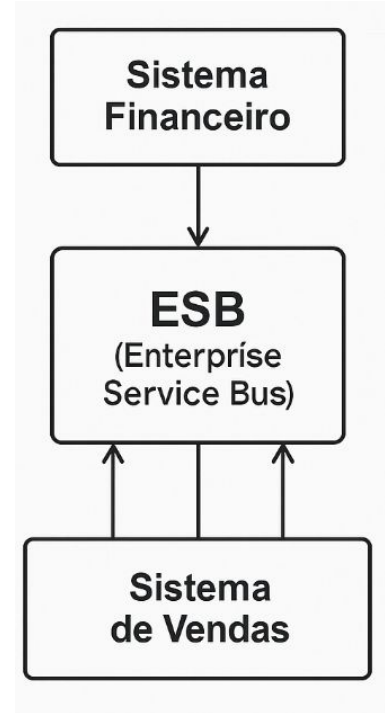
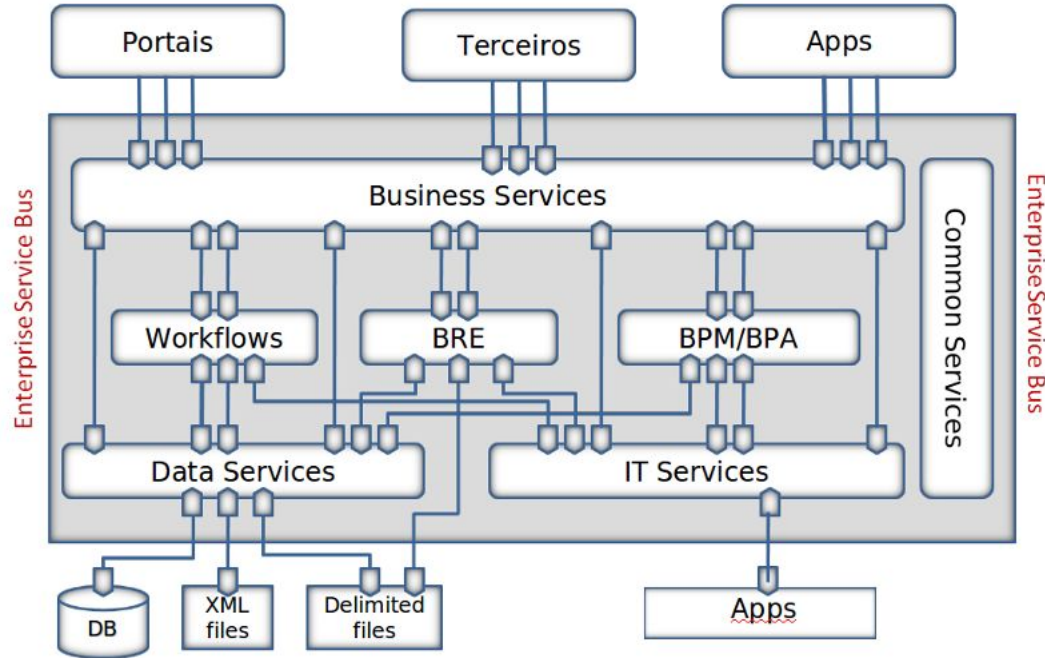
Dimensões do Reuso

- Reuso por Biblioteca: módulos, packages, SDKs
- Reuso por Serviço: APIs REST, SOAP, GraphQL
- Reuso por Arquitetura: SOA e Microserviços
- Reuso por Plataforma: SaaS, PaaS, FaaS

SOA – Service-Oriented Architecture

- SOA é uma arquitetura baseada em serviços corporativos reutilizáveis, que padronizam processos de negócio.
- Características
 - Serviços grandes e corporativos
 - Contratos rígidos (geralmente WSDL + SOAP)
 - Governança centralizada
 - Reuso alto, acoplamento moderado
 - Comunicação via barramento (ESB)

Diagrama SOA





Principais Atores em SOA

- **Provedor de Serviços:** Implementa, mantém e disponibiliza serviços. Garante disponibilidade, segurança e publica descrições dos serviços no Registro.
- **Consumidor de Serviços:** Utiliza os serviços para executar operações. Pode ser uma aplicação, componente ou outro serviço. Encontra os serviços por meio do Registro.
- **Registro de Serviços:** Diretório centralizado contendo descrições dos serviços. Facilita descoberta, integração e reutilização, permitindo interoperabilidade no ecossistema SOA.

Segurança em SOA

- Envolve múltiplas camadas de proteção, incluindo:
 - Autenticação e autorização;
 - Criptografia de dados em trânsito;
 - Políticas de segurança, auditoria e monitoramento;
 - Protocolos seguros (ex.: HTTPS) e uso de tokens, autenticação multifator, etc.
- O objetivo é garantir integridade, confidencialidade e acesso controlado aos serviços e dados.

Pontos fortes e fracos

- Vantagens
 - Reuso forte entre áreas da empresa
 - Governança central ajuda padronização
 - Serviços encapsulam processos críticos
- Desvantagens
 - Alto acoplamento organizacional
 - Evolução mais lenta
 - ESB vira gargalo

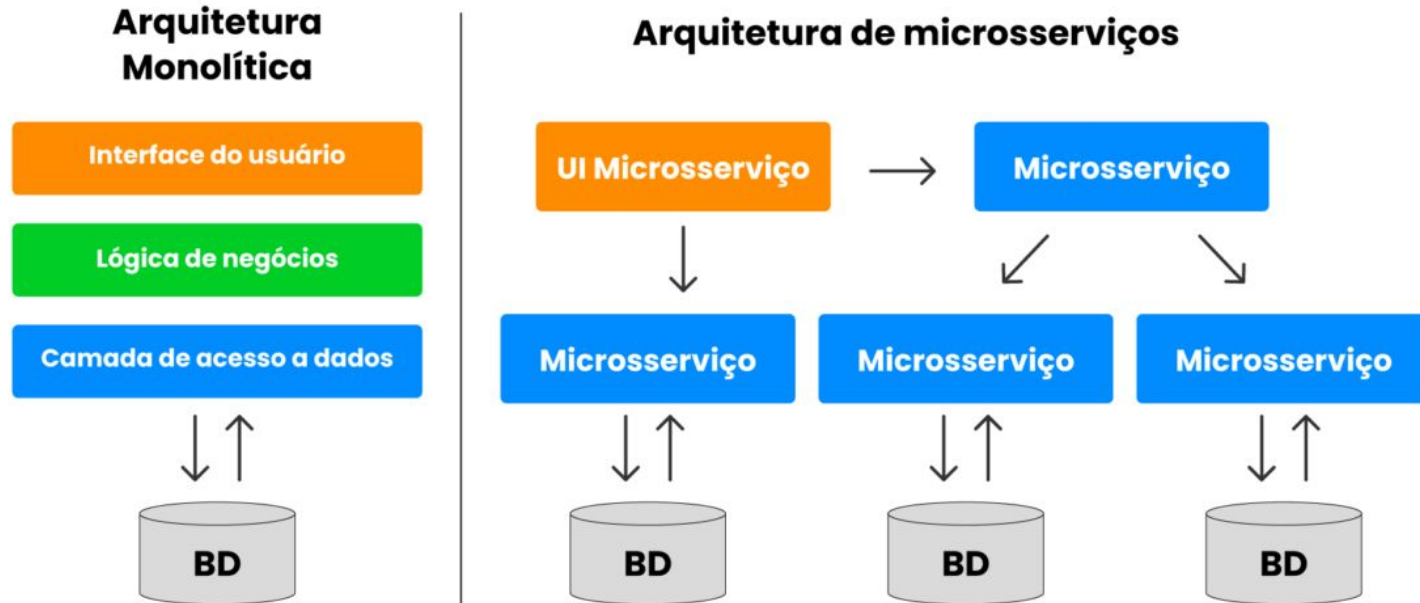
Aplicações e Casos de Uso de SOA

- **Finanças:** Integra sistemas bancários, pagamentos e gestão de riscos, permitindo serviços unificados, personalizados e mais eficientes.
- **Saúde:** Conecta EHR, gestão hospitalar, telemedicina e seguros. Facilita o compartilhamento seguro de informações e melhora o atendimento ao paciente.
- **E-commerce:** Integra vendas online, inventário, pagamentos e atendimento ao cliente, criando uma experiência de compra fluida e facilmente escalável.
- **Telecomunicações:** Unifica faturamento, gestão de assinantes e serviços de valor agregado. Suporta personalização e operação eficiente da rede.

Arquitetura de Microserviços

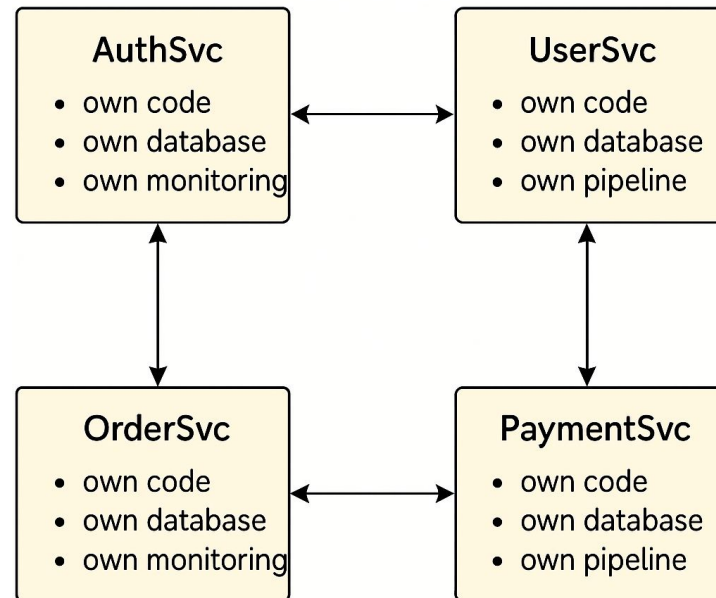
- Arquitetura onde o sistema é dividido em serviços pequenos, independentes e autônomos, cada um responsável por um contexto específico.
- Características
 - Independência total entre serviços
 - Comunicação geralmente via REST/JSON
 - Times pequenos por serviço
 - Deploys isolados
 - Observabilidade integrada (logs, métricas, tracing)

Arquitetura de Microserviços



Diagrama

- Cada serviço possui:
 - código próprio
 - banco de dados próprio
 - monitoramento próprio
 - pipeline de deploy próprio



AuthSvc (Serviço de Autenticação)

- Responsável por:
 - gestão de tokens (JWT, OAuth2),
 - login/logout,
 - verificação de permissões.
- Características:
 - Mantém seu próprio banco com credenciais/identidades.
 - Áreas críticas: performance, segurança e auditoria.
 - Interage com: UserSvc.

UserSvc (Serviço de Usuários)

- Responsável por:
 - dados cadastrais,
 - perfis,
 - preferências de usuários.
- Características:
 - Não armazena senhas (apenas o AuthSvc faz isso).
 - Pode notificar AuthSvc quando houver alteração relevante.
- Interação:
 - AuthSvc <-> UserSvc (ex.: validação de usuário ativo).

OrderSvc (Serviço de Pedidos)

- Responsável por:
 - criação de pedidos,
 - cálculo de frete, totais e estado do pedido,
 - histórico do cliente.
- Características:
 - Banco orientado a eventos ou relacional, dependendo da modelagem.
 - Depende do PaymentSvc durante criação do pedido.
- Interação:
 - OrderSvc <-> PaymentSvc (status de pagamento).

PaymentSvc (Serviço de Pagamentos)

- Responsável por:
 - autorização e captura de pagamento,
 - integração com gateways (Stripe, PayPal, bancos),
 - conciliação.
- Características:
 - Banco próprio contendo transações e logs financeiros.
 - Comunicação geralmente assíncrona (eventos).

Os 12 Factors (Princípios de Aplicações Cloud-Native)

1. Codebase	7. Dependencies
2. Config	8. Backing services
3. Release & Run	9. Processes
4. Port Binding	10. Concurrency
5. Disposability	11. Dev/Prod parity
6. Logs	12. Admin processes

Os 12 Factors (Princípios de Aplicações Cloud-Native)

- 1. Base de código única
 - Uma aplicação → um repositório.
 - Várias deploys podem existir, mas todas vêm do mesmo código-fonte.
- 2. Dependências explícitas
 - Toda dependência deve ser declarada (requirements, pom.xml etc.).
 - O serviço não “assume” nada instalado no sistema.
- 3. Configurações fora do código
 - Configuração não fica hardcoded.
 - Tudo é fornecido via variáveis de ambiente (env vars).
- 4. Serviços de suporte como recursos anexados
 - Bancos de dados, filas, cache → são plugáveis.
 - Podem ser substituídos sem alterar código.

Os 12 Factors (Princípios de Aplicações Cloud-Native)

- 5. Build, Release e Run separados
 - Três etapas distintas:
 - Build: compila e empacota
 - Release: junta código + config
 - Run: executa o serviço
 - Isso permite reprodução, rollback e automação.
- 6. Processos como primeira classe
 - Serviços são stateless.
 - Nada de sessão ou cache local que dependa de um processo específico.
- 7. Bind de porta (Port Binding)
 - “Self-contained”: o serviço expõe sua própria porta (HTTP, gRPC etc.).
 - Nada depende de um servidor externo como Apache/Tomcat embutido.
- 8. Concorrência
 - Escala horizontal.
 - Uma instância para cada tipo de processo (web, worker, consumer etc.).

Os 12 Factors (Princípios de Aplicações Cloud-Native)

- 9. Descartabilidade
 - Processos devem iniciar rápido e morrer rápido.
 - Facilita:
 - escalabilidade,
 - deploys,
 - resiliência.
- 10. Paridade Dev/Prod
 - Dev $\approx$ Prod (o mais próximo possível).
 - Uso de containers, IaC e automação reduz “funciona na minha máquina”.
- 11. Logs como fluxo de eventos
 - Logs são fluxos contínuos enviados a sistemas externos.
 - O serviço não gerencia armazenamento — apenas escreve.
- 12. Processos administrativos isolados
 - Tarefas como migrações, scripts e rotinas são processos à parte.
 - Nunca devem rodar dentro do processo principal da aplicação.

Comparação SOA × Microserviços

Característica	SOA	Microserviços
Granularidade	Grossa	Fina
Comunicação	SOAP, ESB	REST, Mensageria
Governança	Centralizada	Distribuída
Reuso	Muito alto	Modular e independente
Deploy	Dependente	Independente
Acoplamento	Moderado	Baixo

Por que resiliência importa?

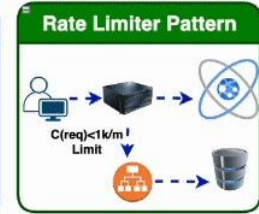
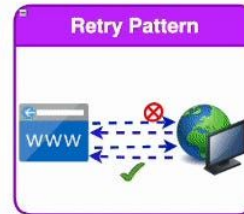
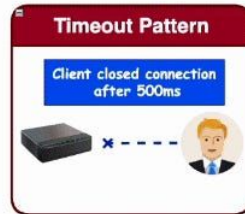
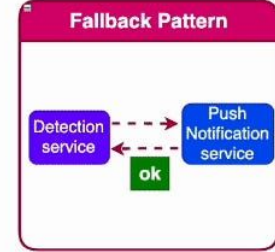
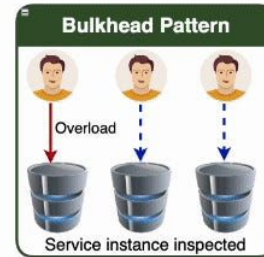
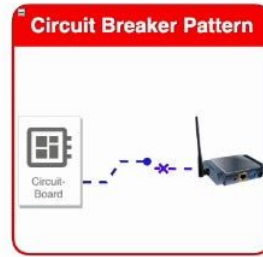
- Em arquiteturas distribuídas (SOA e microsserviços), falhas não são exceção — são a regra.
- Tipos de falhas comuns:
 - Timeout de rede: Pacotes atrasam ou se perdem.
 - Latência alta: Um serviço responde, mas muito lentamente.
 - Serviço fora do ar: Deploy, crash, reinício do pod, falha de hardware.
 - Sobrecarga: Muitas requisições simultâneas saturam o sistema.
 - Falha parcial: O serviço responde para alguns clientes, mas não para todos.
 - Erros intermitentes: Flutuações temporárias — comuns em ambientes cloud.

O que a resiliência garante

- Continuidade do serviço — o sistema segue funcionando mesmo com partes quebrando.
- Estabilidade — evita variação brusca de tempos de resposta.
- Prevenção de efeito cascata — uma falha não derruba o sistema inteiro.
- Melhor experiência do usuário — falhas são tratadas de forma elegante.
- Fail fast — identificar e bloquear rapidamente rotas problemáticas.

Padrões de Resiliência em Microserviços

- Retry
- Timeout
- Circuit Breaker
- Bulkhead
- Fallback
- Rate Limiting
- Saga



Retry (Repetição)

- Repetir uma operação que pode ter falhado por um problema temporário.
Usado para falhas transitórias:
 - perda momentânea de conexão
 - latência rápida
 - pod reiniciando no Kubernetes
 - indisponibilidade curta de banco ou cache

- Boas práticas:
 - Exponential Backoff (aumentar o tempo entre tentativas)
 - Jitter (variação aleatória para evitar sincronização entre clientes)
 - Máximo de tentativas
 - Não usar retry em erros permanentes (ex.: 400, validação)

Exemplo em Python (tenacity)

```
import requests

from tenacity import retry, stop_after_attempt, wait_exponential, retry_if_exception_type

@retry(
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=1, min=1, max=10),
    retry=retry_if_exception_type(requests.exceptions.RequestException)
)

def call_api():
    print("Tentando chamar o serviço...")
    return requests.get("http://localhost:8000/payments").json()

print(call_api())
```

Exemplo em Node.js (axios-retry)

```
const axios = require("axios");
const axiosRetry = require("axios-retry");

axiosRetry(axios, {
  retries: 5,
  retryDelay: axiosRetry.exponentialDelay
});

async function getPayments () {
  return await axios.get("http://localhost:3000/payments" );
}

getPayments ();
```

Timeout (Tempo Limite)

- Evita que requisições fiquem presas indefinidamente.
- Sem timeout → threads travam → fila cresce → serviço morre.
- Por padrão:
 - APIs REST devem ter timeout entre 1 e 5 segundos
 - Chamadas internas entre microserviços devem ser curtas

Exemplos

- Exemplo em Python
 - `requests.get("http://service-b/api", timeout=2)`
- Exemplo no axios
 - `axios.get("http://service-b/api", { timeout: 2000 })`

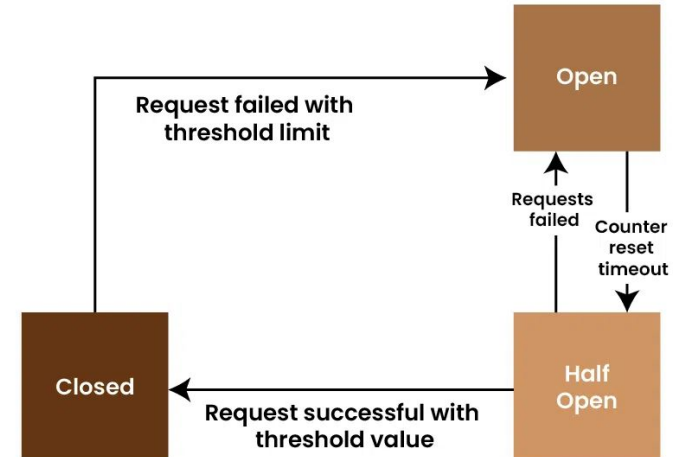


Circuit Breaker

- Um disjuntor lógico que protege seu serviço de continuar chamando um componente que está falhando.
- Ele:
 - detecta falhas,
 - abre o circuito,
 - interrompe chamadas por um tempo,
 - testa periodicamente se o serviço voltou.

Estados do Circuit Breaker

- [CLOSED] → operações normais
- [OPEN] → falhas detectadas, chamadas bloqueadas
- [HALF-OPEN] → tenta algumas chamadas para verificar recuperação



Exemplo em Python (máquina de estados)

```
class CircuitBreaker:
    def __init__(self):
        self.failures = 0
        self.state = "CLOSED"

    def call(self, fn):
        if self.state == "OPEN":
            raise Exception("Circuit breaker aberto!")

        try:
            result = fn()
            self.failures = 0
            return result
        except:
            self.failures += 1
            if self.failures >= 3:
                self.state = "OPEN"
            raise
```

Exemplo em Spring Boot (Resilience4j)

```
@CircuitBreaker(name = "userService", fallbackMethod = "fallbackUsers")

public List<User> listUsers() {
    return restTemplate.getForObject("http://USER-SERVICE/users",
List.class);
}

public List<User> fallbackUsers(Exception e) {
    return List.of(new User("fallback", "User Service offline"));
}
```

Bulkhead (Antepara / Compartimentos)

- Evita que um serviço consuma todos os recursos.
- Ex.: PaymentSvc lento → não pode roubar todas as threads do servidor.
- Estratégias:
 - pools de threads independentes
 - filas separadas
 - limites de concorrência por operação

Exemplo Spring Boot — Bulkhead

YAML

```
resilience4j.bulkhead:
```

```
  instances:
```

```
    paymentBulkhead:
```

```
      maxThreadPoolSize: 10
```

```
      coreThreadPoolSize: 5
```

```
      queueCapacity: 20
```

```
@Bulkhead(name = "paymentBulkhead", type = Bulkhead.Type.THREADPOOL)
```

```
public PaymentResponse processPayment () {
```

```
    return paymentClient.call();
```

```
}
```

Exemplo em Python (asyncio)

```
import asyncio

from asyncio import Semaphore

bulkhead = Semaphore(3)

async def process_order(order_id):
    async with bulkhead:
        print(f"Processando {order_id}")
        await asyncio.sleep(2)

asyncio.run(asyncio.gather(*(process_order(i) for i in range(10))))
```

Fallback (Resposta Alternativa)

- Quando tudo falha → devolver uma resposta útil.
- Exemplos de fallback:
 - resposta cacheada
 - mensagem amigável
 - resposta padrão
 - fila para processar depois (async fallback)

Exemplo Spring (com Circuit Breaker + fallback)

```
public List<User> fallbackUsers(Exception e) {  
    return List.of(new User("fallback", "Serviço temporariamente  
indisponível"));  
}
```

Exemplo Python

```
def get_users():
    try:
        return requests.get(url, timeout=1).json()
    except:
        return {"users": [], "message": "Serviço offline, usando
fallback"}
```

Rate Limiting (Limitação de Taxa)

- Evita sobrecarga de APIs limitando o número de requisições por cliente.
- Usado por:
 - gateways (Kong, NGINX, Traefik)
 - APIs públicas
 - sistemas de billing
- Estratégias comuns:
 - Token Bucket
 - Leaky Bucket
 - Sliding Window
 - Fixed Window

Exemplo em Node.js (express-rate-limit)

```
const rateLimit = require("express-rate-limit");

const limiter = rateLimit({
  windowMs: 60 * 1000,
  max: 100
});

app.use("/api/", limiter);
```

Exemplo em NGINX

```
limit_req_zone $binary_remote_addr zone=mylimit:10m rate=5r/s;

server {
    location /api/ {
        limit_req zone=mylimit burst=10;
    }
}
```

Saga Pattern (Transações Distribuídas)

- Resolve transações distribuídas sem 2PC. Divide uma transação longa em etapas com ações compensatórias.
- Exemplo de e-commerce:
 - Criar pedido
 - Debitar estoque
 - Cobrar pagamento
- Se cobrança falhar, então:
 - desfazer cobrança
 - devolver estoque
 - cancelar pedido
- Modelos de Saga:
 - Orquestrada (um coordenador central)
 - Coreografada (serviços reagem a eventos)

Exemplo estilo coreografia (eventos)

- OrderSvc:
 - { "event": "ORDER_CREATED", "orderId": 123 }
- InventorySvc:
 - { "event": "STOCK_RESERVED", "orderId": 123 }
- PaymentSvc:
 - { "event": "PAYMENT_FAILED", "orderId": 123 }
- OrderSvc compensação:
 - { "event": "ORDER_CANCELLED", "orderId": 123 }

Resumo

Padrão	Problema que resolve	Resultado
Retry	falhas momentâneas	mais robustez
Timeout	chamadas travadas	menos bloqueios
Circuit Breaker	falhas repetidas	evita cascata
Bulkhead	esgotamento de recursos	isolamento
Fallback	serviço indisponível	resposta útil
Rate Limiting	excesso de requisições	protege API
Saga	transações distribuídas	consistência eventual

Microserviço REST (FastAPI) → Conversor de moeda

GET /convert Converter

Parameters Cancel

Name	Description
valor * required number (query)	23
moeda_origem * required string (query)	BRL
moeda_destino * required string (query)	EUR

Execute Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/convert?valor=23&moeda_origem=BRL&moeda_destino=EUR' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/convert?valor=23&moeda_origem=BRL&moeda_destino=EUR
```

Server response

Code	Details
200	Response body <pre>{ "valor": 23, "de": "BRL", "para": "EUR", "valor_convertido": 4.22 }</pre> Download

Dependências

- pip install
 - fastapi
 - uvicorn
 - httpx

Funcionalidades implementadas

- Retry Pattern
 - Requisições à API externa tentam novamente com exponential backoff.
- Circuit Breaker
 - Se muitas falhas ocorrerem, o sistema abre o circuito e para de chamar a API externa por um tempo configurado.
- Bulkhead (Isolamento)
 - Limite máximo de requisições simultâneas à API externa usando um Semaphore.
- LRU Cache real
 - Cache baseado em `functools.lru_cache` para armazenar as taxas por 10 minutos.

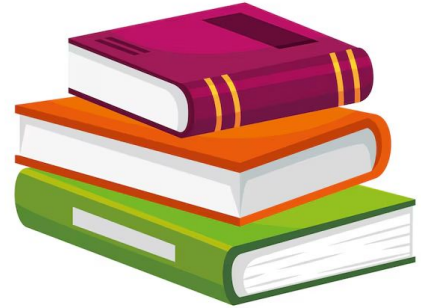
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

